

Deep Linking Integration for Mobile Applications

Complete Research & Implementation Document

Project: ecoATM Mobile App

Prepared: April 2026

Status: Spike / Pre-implementation

Table of Contents

1. [What is Deep Linking?](#)
 2. [App Identifiers & Store URLs](#)
 3. [Core Architecture & Decision Flow](#)
 4. [The Two Scheme Layers](#)
 5. [What to Prepare Before You Start](#)
 6. [Channel Requirements & Link Samples](#)
 7. [Hosting Infrastructure \(Netlify\)](#)
 8. [Website Implementation](#)
 9. [Android App Implementation](#)
 10. [iOS App Implementation](#)
 11. [Testing & Verification](#)
 12. [Troubleshooting Reference](#)
 13. [Open Questions & Dependencies](#)
-

1. What is Deep Linking?

Deep linking is the practice of using a URL to open a specific screen inside a mobile app, rather than just launching the app to its home screen. When a user taps a link from any channel (email, SMS, push notification, social media, or QR code), the system determines:

1. Is the user on iOS or Android?
2. Is the app installed on their device?
3. If yes → open the app at the correct screen
4. If no → redirect to the App Store or Google Play

There are two technical approaches, and this project uses **both** for different reasons (explained in Section 4):

- **HTTPS Universal Links / App Links** — the modern, verified, preferred method
 - **Custom URI schemes** (`ecoatm://`) — the fallback method for push and in-app browsers
-

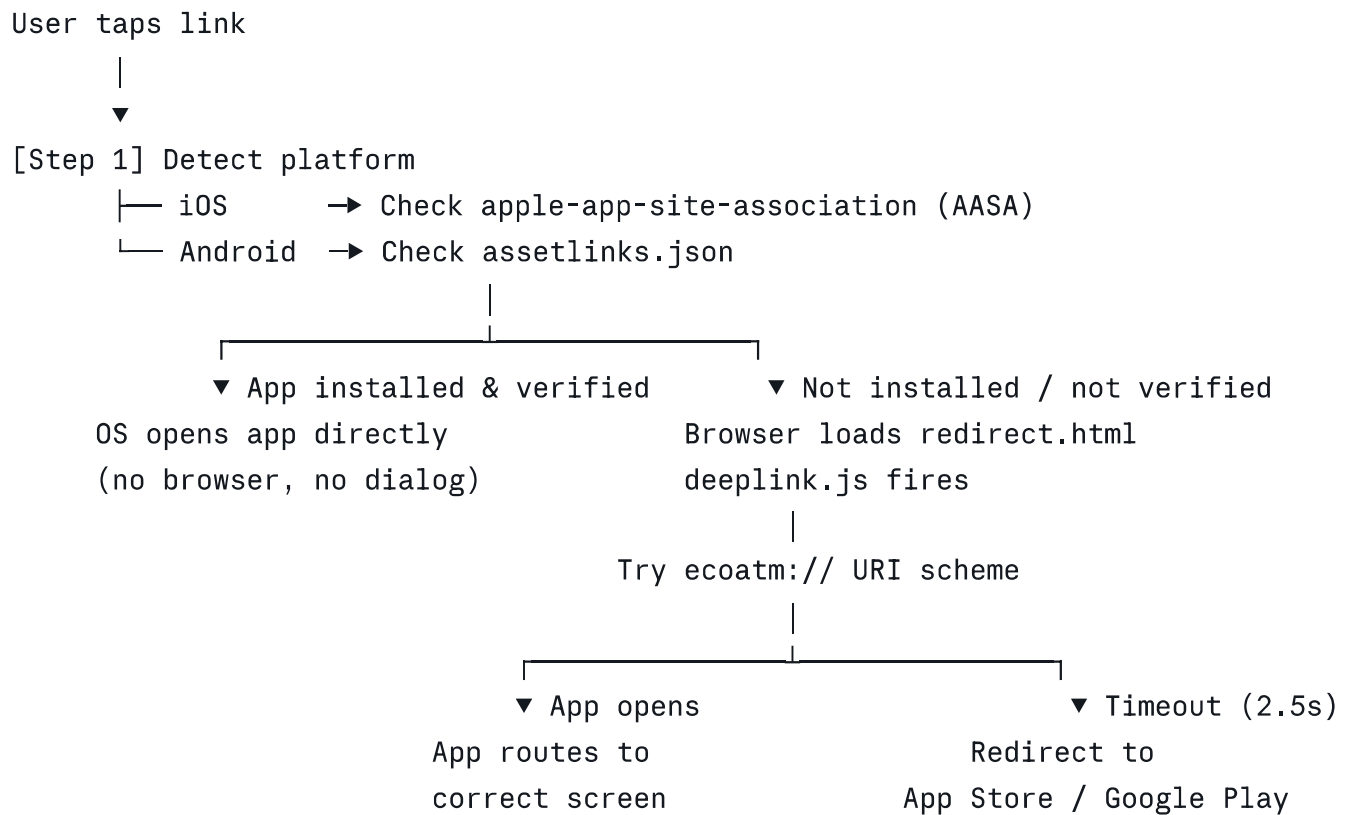
2. App Identifiers & Store URLs

Platform	Bundle / Application ID	Store URL
Android	<code>com.ecoatm.ecoapp.android_qa</code>	https://play.google.com/store/apps/details?id=com.ecoatm.ecoapp.android
iOS	<code>com.outerwall.ecoATM2</code>	https://apps.apple.com/us/app/ecocatm/id944835823

Note: The Android bundle ID used throughout this document is the QA variant (`android_qa`). Confirm the production bundle ID before go-live and update `assetlinks.json` and `AndroidManifest.xml` accordingly.

3. Core Architecture & Decision Flow

Every deep link — regardless of channel — routes through the same decision logic:



Key principle: The OS-level interception (App Links / Universal Links) happens *before* any JavaScript runs. If verification is correctly set up, the user never sees the browser or the redirect page — the app opens instantly. JavaScript is only the fallback.

4. The Two Scheme Layers

This is one of the most important architectural decisions in this project. Both schemes are required — they serve different roles.

Layer 1: HTTPS App Links / Universal Links (Primary)

Property	Value
iOS scheme	<code>https://</code> with AASA verification
Android scheme	<code>https://</code> with <code>assetlinks.json</code> verification
Manifest flag	<code>android:autoVerify="true"</code>
Verification file	<code>.well-known/apple-app-site-association</code> and <code>.well-known/assetlinks.json</code>

Property	Value
Channels handled	Email, SMS, Social Media, QR Code
Security	Cryptographically tied to your signing certificate — only your app can claim the domain
Behavior	OS intercepts the URL before the browser loads; no disambiguation dialog

Layer 2: Custom URI Scheme — `ecoatm://` (Fallback)

Property	Value
Scheme	<code>ecoatm://</code>
URI format	<code>ecoatm://screen/{screenName}?{params}</code>
No verification required	Any app can register a custom scheme — works immediately
Channels handled	Push notifications, In-app browser JS fallback (Instagram, Facebook, TikTok)
Risk	Any app can claim <code>ecoatm://</code> — never use as primary mechanism
Behavior	Disambiguation dialog may appear if multiple apps claim the same scheme

Why NOT to replace HTTPS with `ecoatm://` only

- Gmail, Outlook, and many email clients **block custom URI schemes** entirely
- Custom schemes are **deprecated as primary deep links in Android 12+**
- No ownership verification — **any app can hijack** your scheme
- HTTPS links survive carrier link scanning; custom URIs do not

Decision table by channel

Channel	Primary scheme	Fallback scheme
Email	HTTPS App Link	<code>ecoatm://</code> (via redirect page)
SMS	HTTPS App Link	<code>ecoatm://</code> (via redirect page)
Social (native browser)	HTTPS App Link	<code>ecoatm://</code> (via redirect page)

Channel	Primary scheme	Fallback scheme
Social (in-app browser)	ecoatm:// via JS	Store redirect
QR Code	HTTPS App Link	ecoatm:// (via redirect page)
Push notification	ecoatm:// in payload	None (app already installed)

5. What to Prepare Before You Start

5.1 Information to collect

Item	Where to find it	Used in
Apple Team ID	developer.apple.com → Membership	<code>apple-app-site-association</code>
iOS Bundle ID	Xcode → project settings	<code>apple-app-site-association</code> , iOS manifest
Android package name	<code>build.gradle</code> → <code>applicationId</code>	<code>assetlinks.json</code> , AndroidManifest
Android SHA-256 signing certificate	<code>keytool -list -v -keystore release.jks</code>	<code>assetlinks.json</code>
iOS App Store ID	App Store Connect or store URL (<code>id944835823</code>)	Store fallback URL
Android Play Store URL	Google Play Console	Store fallback URL
Deep link domain	Your hosting domain	All links, AASA, assetlinks
App screen route names	Mobile engineering	<code>DeepLinkHandler.kt</code> / <code>DeepLinkHandler.swift</code>

5.2 Get the Android SHA-256 fingerprint

```
bash
```

```
# From your keystore file:
keytool -list -v -keystore your-release.jks -alias your-key-alias

# Or from an already-built APK:
keytool -printcert -jarfile your-app.apk

# Output – copy the SHA256 line:
# SHA256: AB:CD:EF:01:23:45:67:89:...
```

5.3 Get the Apple Team ID

1. Log in to developer.apple.com
2. Navigate to **Account** → **Membership**
3. Copy the **Team ID** (10-character alphanumeric string, e.g. `ABC123DEF4`)

5.4 Infrastructure checklist

- ☐ Domain registered and DNS pointing to Netlify (`links.ecoatm.com`)
 - ☐ TLS certificate active (Netlify provisions automatically)
 - ☐ `apple-app-site-association` accessible at `https://links.ecoatm.com/.well-known/apple-app-site-association`
 - ☐ `assetlinks.json` accessible at `https://links.ecoatm.com/.well-known/assetlinks.json`
 - ☐ Both files return `Content-Type: application/json` with no redirects
 - ☐ App released or in TestFlight/internal testing with the correct signing certificate
-

6. Channel Requirements & Link Samples

All HTTPS deep links follow this URL structure:

```
https://links.ecoatm.com/{channel}?screen={screen}&{optional_params}&utm_source={source}&utm_medium={medium}&utm_campaign={campaign}
```

6.1 Email

Requirement: HTTPS Universal Link / App Link

Why HTTPS is mandatory: Gmail, Outlook, and Apple Mail strip or sandbox custom URI schemes. HTTPS links pass safely through all major email clients.

Sample link:

```
https://links.ecoatm.com/email?screen=trade-  
in&promo=SPRING25&utm_source=email&utm_medium=cta&utm_campaign=spring_promo
```

What marketing needs:

- The ESP (Braze, Iterable, Salesforce MC) must wrap the link using the deep link service's click-tracking wrapper — not a raw HTTPS URL
- Each campaign CTA should have its own `utm_campaign` value for attribution

Email client compatibility:

Client	Behavior	Handled by
Gmail (mobile)	Wraps link in <code>google.com/url?q=</code> redirect	Deep link redirect page survives this
Apple Mail	Fully supports Universal Links	No special handling needed
Outlook (mobile)	Opens in in-app browser	JS fallback on redirect page
Braze / Iterable / SFMC	May URL-encode or wrap links	Use platform's native link wrapper

Fallback: App not installed → App Store / Google Play

6.2 SMS

Requirement: Short HTTPS link

Why short: SMS has a 160-character limit per segment. Long URLs with UTM parameters fragment into multi-part messages, incurring additional carrier costs.

Sample link (short):

```
https://eco.atm/s/k9Xp2
```

Resolves to:

```
https://links.ecoatm.com/sms?  
screen=sell&utm_source=sms&utm_medium=text&utm_campaign=winback
```

Important carrier rules:

- Use a **branded short domain** (e.g. `eco.atm/s/`) — not a generic shortener like bit.ly. Generic shorteners are frequently on carrier spam blocklists and can silently break the redirect chain
- Open Graph preview metadata is irrelevant in SMS — suppress it
- Do **not** use custom URI schemes in SMS — most carriers block them

Fallback: App not installed → App Store / Google Play

6.3 Push Notifications

Requirement: `ecoatm://` URI delivered directly in the FCM/APNS push payload

Why no web fallback: Push is only delivered to users who have already installed the app. There is no scenario where the app is absent.

Sample payload (FCM):

```
json
{
  "notification": {
    "title": "Your device is worth $85!",
    "body": "Tap to see your offer before it expires."
  },
  "data": {
    "deep_link": "ecoatm://screen/offer?offerId=0FR-2938&utm_source=push&utm_medium="
  }
}
```

iOS handler: `UNUserNotificationCenterDelegate` →

`userNotificationCenter(_:didReceive:withCompletionHandler:)`

Android handler: `FirebaseMessagingService.onMessageReceived()`

Critical: Route the deep link *after* app launch/resume is complete to avoid a race condition on cold start.

6.4 Social Media (Paid & Organic)

Requirement: HTTPS Universal Link + Open Graph metadata

Why OG metadata: Facebook, Instagram, TikTok, Twitter/X, and LinkedIn unfurl rich link previews. Without OG tags the preview is blank.

Sample link:

```
https://links.ecoatm.com/social?
screen=sell&utm_source=instagram&utm_medium=paid_social&utm_campaign=q2_awareness
```

Required Open Graph tags (on the redirect page):

```
html

<meta property="og:title"      content="ecoATM – Sell Your Phone for Instant Cash">
<meta property="og:description" content="Get an instant offer at your nearest ecoATM">
<meta property="og:image"     content="https://links.ecoatm.com/assets/og-social.jpg">
<meta property="og:url"       content="https://links.ecoatm.com/social?screen=sell">
<meta property="og:type"      content="website">
```

OG image spec: 1200 × 630px, max 20% text (Facebook paid ad requirement), min 600 × 315px

In-App Browser (IAB) problem:

Instagram, Facebook, and TikTok open links in their own in-app browsers, which intentionally block Universal Link / App Link interception. The JavaScript redirect fallback handles this:

```
User taps link in Instagram IAB
|
▼
redirect.html loads in IAB
|
JS detects: Universal Links blocked (IAB)
|
window.location = "ecoatm://screen/sell"
|
├─ App installed → app opens
└─ No response in 2.5s → redirect to App Store
```

IAB compatibility by platform:

Platform	IAB	Universal Links	Mitigation
Instagram	Yes	Blocked	JS + URI scheme fallback
Facebook	Yes	Blocked	JS + URI scheme fallback
TikTok	Yes	Blocked	JS + URI scheme fallback
Twitter/X	Partial	Sometimes works	JS fallback as safety net
LinkedIn	No (native browser)	Works natively	None needed

--

6.5 QR Codes

Requirement: HTTPS deep link encoded into a QR code image

Use case: Physical marketing — in-store kiosk signage, print ads, mailers, packaging inserts

Sample link (encoded into QR):

```
https://links.ecoatm.com/qr?
screen=sell&utm_source=qr&utm_medium=in_store&utm_campaign=kiosk_signage
```

QR code specifications:

Property	Requirement	Reason
Error correction	Level H (30%)	Tolerates physical wear, partial obstruction, low print quality
Minimum print size	1 × 1 inch at 300 dpi	Scannable at arm's length
Quiet zone	4 module widths on all sides	Required by QR standard
Colors	Dark modules on white/light background	Inverted QR codes fail on many readers
File format	SVG for print, PNG @2x for digital	SVG scales without quality loss
URL length	Use full HTTPS URL — do not shorten	Short codes reduce scan reliability on degraded print

Each placement (kiosk, print ad, mailer) should use a unique `utm_campaign` value so the marketing team can attribute installs per physical location.

6.6 Screen Route Reference

<code>screen</code> value	App destination
<code>home</code>	App home screen
<code>sell</code>	Start sell / trade-in flow
<code>trade-in</code>	Trade-in flow entry (accepts <code>promo</code> param)
<code>offer</code>	Specific offer (requires <code>offerId</code> param)
<code>find-kiosk</code>	Kiosk locator map
<code>account</code>	User account / profile

7. Hosting Infrastructure (Netlify)

Why Netlify over GitHub Pages

Feature	GitHub Pages	Netlify (free)
<code>_redirects</code> rules	Not supported	Native
Custom response headers (<code>_headers</code>)	Not supported	Native
<code>Content-Type: application/json</code> for AASA (no extension)	Requires workaround	Works out of the box
Edge functions (dynamic OG tags)	No	Free tier
TLS certificate	Auto	Auto
Deploy from drag & drop	No	Yes
Custom domain	Yes	Yes

The `_headers` file is what makes AASA and `assetlinks.json` return `Content-Type: application/json`. Without this, both iOS and Android silently fail verification. GitHub Pages cannot set this header for extensionless files without a workaround.

File structure

```
ecoatm-deeplink/
├─ index.html           ← Marketing landing page
├─ redirect.html        ← Deep link router (all channels)
├─ _redirects           ← Netlify redirect rules
├─ _headers             ← Netlify custom headers
├─ .well-known/
│   ├─ apple-app-site-association ← iOS Universal Links config
│   └─ assetlinks.json    ← Android App Links config
├─ css/
│   └─ style.css
└─ js/
    └─ deeplink.js       ← Device detection + routing logic
```

`_redirects` rules

```
/email      /redirect.html  200
/sms        /redirect.html  200
/push       /redirect.html  200
/social     /redirect.html  200
/qr         /redirect.html  200
/s/:code    /redirect.html?ref=:code  200
/*          /index.html    200
```

`_headers` rules

```
/.well-known/apple-app-site-association
  Content-Type: application/json
  Access-Control-Allow-Origin: *

/.well-known/assetlinks.json
  Content-Type: application/json
  Access-Control-Allow-Origin: *

/*
  X-Frame-Options: DENY
```

X-Content-Type-Options: nosniff

Strict-Transport-Security: max-age=63072000; includeSubDomains; preload

8. Website Implementation

8.1 `apple-app-site-association`

```
json
{
  "applinks": {
    "details": [
      {
        "appIDs": ["TEAMID.com.outerwall.ecoATM2"],
        "components": [
          { "/": "/redirect*" },
          { "/": "/email*" },
          { "/": "/sms*" },
          { "/": "/social*" },
          { "/": "/qr*" },
          { "/": "/s/*" }
        ]
      }
    ]
  }
}
```

Replace `TEAMID` with your 10-character Apple Team ID.

8.2 `assetlinks.json`

```
json
```

```
[
  {
    "relation": ["delegate_permission/common.handle_all_urls"],
    "target": {
      "namespace": "android_app",
      "package_name": "com.ecoatm.ecoapp.android_qa",
      "sha256_cert_fingerprints": [
        "AB:CD:EF:01:23:..."
      ]
    }
  }
]
```

Replace the fingerprint with your actual SHA-256 signing certificate value.

8.3 `deeplink.js` — core redirect logic

The JavaScript router on `redirect.html` performs three steps:

Step 1 — Platform detection:

```
javascript

function getPlatform() {
  const ua = navigator.userAgent || '';
  if (/iPhone|iPad|iPod/i.test(ua)) return 'ios';
  if (/Android/i.test(ua))         return 'android';
  return 'desktop';
}
```

Step 2 — In-app browser detection:

```
javascript

function isInAppBrowser() {
  const ua = navigator.userAgent || '';
  return /FBAN|FBAV|Instagram|Twitter|LinkedInApp|TikTok/i.test(ua);
}
```

Step 3 — Android Intent URI (recommended over iframe trick):

```
javascript
```

```
function buildIntentUri(appUri, storeUrl) {  
  const withoutScheme = appUri.replace('ecoatm://', '');  
  return (  
    'intent://' + withoutScheme +  
    '#Intent' +  
    ';scheme=ecoatm' +  
    ';package=com.ecoatm.ecoapp.android_qa' +  
    ';S.browser_fallback_url=' + encodeURIComponent(storeUrl) +  
    ';end'  
  );  
}
```

The Intent URI is the correct approach for Android Chrome. It hands the install-check decision to the Android OS: if the package is installed, the OS opens it directly; if not, it follows `S.browser_fallback_url` to the Play Store — no JavaScript timeout is needed to detect install status on Chrome.

9. Android App Implementation

9.1 `AndroidManifest.xml`

Two intent filters are required in the launcher Activity:

xml

```

<activity
    android:name=".MainActivity"
    android:exported="true"
    android:launchMode="singleTask">

    <!-- Standard launcher -->
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>

    <!--
    HTTPS App Links – primary deep link mechanism.
    android:autoVerify="true" triggers assetlinks.json verification at install time
    Option A: catch-all pattern (simpler)
    -->
    <intent-filter android:autoVerify="true">
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="https"
            android:host="links.ecoatm.com"
            android:pathPattern=".*" />
    </intent-filter>

    <!--
    Custom URI scheme – fallback for push and in-app browsers.
    No autoVerify – works without assetlinks.json.
    -->
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="ecoatm" />
    </intent-filter>

</activity>

```

`android:launchMode="singleTask"` is required. Without it, tapping a deep link while the app is already open creates a second Activity instance, and the link is handled by the wrong instance or dropped entirely.

kotlin

```

object DeepLinkHandler {

    sealed class DeepLinkDestination {
        object Home : DeepLinkDestination()
        object Sell : DeepLinkDestination()
        object FindKiosk : DeepLinkDestination()
        object Account : DeepLinkDestination()
        data class Offer(val offerId: String) : DeepLinkDestination()
        data class TradeIn(val promoCode: String? = null) : DeepLinkDestination()
        object Unknown : DeepLinkDestination()
    }

    fun resolve(intent: Intent?): DeepLinkDestination? {
        if (intent?.action != Intent.ACTION_VIEW) return null
        val uri = intent.data ?: return null
        return when (uri.scheme) {
            "https" -> parseHttpsUri(uri)
            "ecoatm" -> parseCustomUri(uri)
            else -> DeepLinkDestination.Unknown
        }
    }

    private fun parseHttpsUri(uri: Uri): DeepLinkDestination {
        val screen = uri.getQueryParameter("screen") ?: "home"
        val offerId = uri.getQueryParameter("offerId")
        val promo = uri.getQueryParameter("promo")
        return screenToDestination(screen, offerId, promo)
    }

    private fun parseCustomUri(uri: Uri): DeepLinkDestination {
        // ecoatm://screen/sell?offerId=OFR-123
        val screen = uri.pathSegments.firstOrNull() ?: "home"
        val offerId = uri.getQueryParameter("offerId")
        val promo = uri.getQueryParameter("promo")
        return screenToDestination(screen, offerId, promo)
    }

    private fun screenToDestination(screen: String, offerId: String?, promo: String?)
        when (screen.lowercase()) {
            "home" -> DeepLinkDestination.Home
            "sell" -> DeepLinkDestination.Sell
            "trade-in", "tradein" -> DeepLinkDestination.TradeIn(promo)
            "offer" -> offerId?.let { DeepLinkDestination.Offer(it) }
        }
    }
}

```

```

        ?: DeepLinkDestination.Home
        "find-kiosk"      -> DeepLinkDestination.FindKiosk
        "account"         -> DeepLinkDestination.Account
        else              -> DeepLinkDestination.Unknown
    }
}

```

9.3 MainActivity.kt

kotlin

```

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        // Guard against re-routing on rotation / config change
        if (savedInstanceState == null) handleDeepLink(intent)
    }

    // Called when app is already running and a new intent arrives
    override fun onNewIntent(intent: Intent) {
        super.onNewIntent(intent)
        setIntent(intent)
        handleDeepLink(intent)
    }

    private fun handleDeepLink(intent: Intent?) {
        when (DeepLinkHandler.resolve(intent)) {
            is DeepLinkDestination.Home      -> navigateToHome()
            is DeepLinkDestination.Sell       -> navigateToSell()
            is DeepLinkDestination.TradeIn    -> navigateToTradeIn(it.promoCode)
            is DeepLinkDestination.Offer      -> navigateToOffer(it.offerId)
            is DeepLinkDestination.FindKiosk  -> navigateToFindKiosk()
            is DeepLinkDestination.Account    -> navigateToAccount()
            else                              -> navigateToHome()
        }
    }
}

```

Both `onCreate` and `onNewIntent` must handle the intent. `onCreate` covers cold starts; `onNewIntent` covers when the app is already in the foreground or background.

10. iOS App Implementation

10.1 Entitlements file

Add the `com.apple.developer.associated-domains` entitlement to your app target:

```
xml

<!-- ecoATM.entitlements -->
<key>com.apple.developer.associated-domains</key>
<array>
    <string>applinks:links.ecoatm.com</string>
</array>
```

10.2 URL scheme registration (`Info.plist`)

```
xml

<key>CFBundleURLTypes</key>
<array>
    <dict>
        <key>CFBundleURLSchemes</key>
        <array>
            <string>ecoatm</string>
        </array>
        <key>CFBundleURLName</key>
        <string>com.outerwall.ecoATM2</string>
    </dict>
</array>
```

10.3 `AppDelegate.swift` — handle both entry points

```
swift
```

```

import UIKit

@UIApplicationMain
class AppDelegate: UIResponder, UIApplicationDelegate {

    // Cold start via Universal Link
    func application(_ application: UIApplication,
                     continue userActivity: NSUserActivity,
                     restorationHandler: @escaping ([UIUserActivityRestoring]?) -> Void) -> Bool {
        guard userActivity.activityType == NSUserActivityTypeBrowsingWeb,
              let url = userActivity.webpageURL else { return false }
        return DeepLinkHandler.shared.handle(url: url)
    }

    // Custom URI scheme: ecoatm://
    func application(_ app: UIApplication,
                     open url: URL,
                     options: [UIApplication.OpenURLOptionsKey: Any] = [:]) -> Bool {
        return DeepLinkHandler.shared.handle(url: url)
    }
}

```

10.4 `SceneDelegate.swift` — required for iOS 13+

swift

```

class SceneDelegate: UIResponder, UIWindowSceneDelegate {

    // App not running – launched by Universal Link
    func scene(_ scene: UIScene,
               willConnectTo session: UISceneSession,
               options connectionOptions: UIScene.ConnectionOptions) {
        if let userActivity = connectionOptions.userActivities.first,
           userActivity.activityType == NSUserActivityTypeBrowsingWeb,
           let url = userActivity.webpageURL {
            DeepLinkHandler.shared.handle(url: url)
        }
        if let url = connectionOptions.urlContexts.first?.url {
            DeepLinkHandler.shared.handle(url: url)
        }
    }

    // App running – Universal Link arrives
    func scene(_ scene: UIScene, continue userActivity: NSUserActivity) {
        guard userActivity.activityType == NSUserActivityTypeBrowsingWeb,
              let url = userActivity.webpageURL else { return }
        DeepLinkHandler.shared.handle(url: url)
    }

    // App running – custom scheme arrives
    func scene(_ scene: UIScene, openURLContexts urlContexts: Set<UIOpenURLContext>) {
        guard let url = urlContexts.first?.url else { return }
        DeepLinkHandler.shared.handle(url: url)
    }
}

```

10.5 DeepLinkHandler.swift

swift

```

import UIKit

final class DeepLinkHandler {

    static let shared = DeepLinkHandler()
    private init() {}

    enum Destination {
        case home
        case sell
        case tradeIn(promoCode: String?)
        case offer(offerId: String)
        case findKiosk
        case account
        case unknown
    }

    @discardableResult
    func handle(url: URL) -> Bool {
        let destination = resolve(url: url)
        route(to: destination)
        return destination != .unknown
    }

    private func resolve(url: URL) -> Destination {
        switch url.scheme?.lowercased() {
            case "https": return parseHttpsURL(url)
            case "ecoatm": return parseCustomURL(url)
            default:      return .unknown
        }
    }

    private func parseHttpsURL(_ url: URL) -> Destination {
        let components = URLComponents(url: url, resolvingAgainstBaseURL: false)
        let screen = components?.queryItems?.first(where: { $0.name == "screen" })?
        let offerId = components?.queryItems?.first(where: { $0.name == "offerId" })
        let promo = components?.queryItems?.first(where: { $0.name == "promo" })?.
        return screenToDestination(screen, offerId: offerId, promo: promo)
    }

    private func parseCustomURL(_ url: URL) -> Destination {
        // ecoatm://screen/sell?offerId=OFR-123
        let screen = url.pathComponents.dropFirst().first ?? "home"
    }
}

```

```

        let comps    = URLComponents(url: url, resolvingAgainstBaseURL: false)
        let offerId = comps?.queryItems?.first(where: { $0.name == "offerId" })?.value
        let promo    = comps?.queryItems?.first(where: { $0.name == "promo" })?.value
        return screenToDestination(screen, offerId: offerId, promo: promo)
    }

    private func screenToDestination(_ screen: String,
                                     offerId: String?,
                                     promo: String?) -> Destination {
        switch screen.lowercased() {
        case "home":           return .home
        case "sell":           return .sell
        case "trade-in","tradein": return .tradeIn(promoCode: promo)
        case "offer":          return offerId.map { .offer(offerId: $0) } ?? .home
        case "find-kiosk":     return .findKiosk
        case "account":        return .account
        default:               return .unknown
        }
    }

    private func route(to destination: Destination) {
        // Replace with your navigation coordinator / router calls
        switch destination {
        case .home:            navigateToHome()
        case .sell:            navigateToSell()
        case .tradeIn(let promo): navigateToTradeIn(promoCode: promo)
        case .offer(let id):   navigateToOffer(offerId: id)
        case .findKiosk:       navigateToFindKiosk()
        case .account:         navigateToAccount()
        case .unknown:         navigateToHome()
        }
    }

    private func navigateToHome() { /* NavController call */ }
    private func navigateToSell() { /* NavController call */ }
    private func navigateToTradeIn(promoCode: String?) { /* NavController call */ }
    private func navigateToOffer(offerId: String) { /* NavController call */ }
    private func navigateToFindKiosk() { /* NavController call */ }
    private func navigateToAccount() { /* NavController call */ }
}

```


11. Testing & Verification

11.1 Verify the well-known files

```
bash

# Check assetlinks.json – must be 200, no redirects, Content-Type: application/json
curl -Lv https://links.ecoatm.com/.well-known/assetlinks.json

# Check AASA – same requirements
curl -Lv https://links.ecoatm.com/.well-known/apple-app-site-association

# Validate via Google's Digital Asset Links API
curl "https://digitalassetlinks.googleapis.com/v1/statements:list?source.web.site=https://links.ecoatm.com"
# Expect: statements array with your package name

# Validate iOS AASA via Apple's CDN
curl https://app-site-association.cdn-apple.com/a/v1/links.ecoatm.com
```

11.2 Verify Android App Links on device

```
bash

# Check verification status – must show "verified" for links.ecoatm.com
adb shell pm get-app-links --user cur com.ecoatm.ecoapp.android_qa

# Force re-verification (useful after updating assetlinks.json)
adb shell pm set-app-links --package com.ecoatm.ecoapp.android_qa 2 links.ecoatm.com

# Test an HTTPS App Link directly
adb shell am start \
  -a android.intent.action.VIEW \
  -c android.intent.category.BROWSABLE \
  -d "https://links.ecoatm.com/redirect?screen=sell&utm_source=test"

# Test the custom URI scheme
adb shell am start \
  -a android.intent.action.VIEW \
  -d "ecoatm://screen/sell?offerId=OFR-TEST-001"
```

11.3 Full channel test matrix

Test case	Email	SMS	Push	Social	QR
iOS — app installed → correct screen	✓	✓	✓	✓	✓
iOS — app not installed → App Store	✓	✓	N/A	✓	✓
Android — app installed → correct screen	✓	✓	✓	✓	✓
Android — app not installed → Google Play	✓	✓	N/A	✓	✓
Gmail (mobile) URL-wrapping survives	✓	—	—	—	—
Outlook (mobile) in-app browser	✓	—	—	—	—
Instagram IAB → JS fallback	—	—	—	✓	—
Facebook IAB → JS fallback	—	—	—	✓	—
TikTok IAB → JS fallback	—	—	—	✓	—
Desktop → ecoatm.com (not store)	✓	✓	—	✓	✓
UTM params preserved end-to-end	✓	✓	✓	✓	✓
Push cold start — correct screen	—	—	✓	—	—
Push foreground (onNewIntent) — correct screen	—	—	✓	—	—

12. Troubleshooting Reference

Symptom	Most likely cause	Fix
Android redirects to Play Store even when app is installed	<code>assetlinks.json</code> SHA-256 fingerprint mismatch	Re-run <code>keytool</code> and update the fingerprint
Android shows disambiguation dialog	App Links not verified; multiple apps claim the scheme	Fix fingerprint; only use <code>ecoa: //</code> as fallback
iOS opens Safari instead of app	AASA not cached yet, or Team ID wrong	Wait 24h for Apple CDN; verify Team ID

Symptom	Most likely cause	Fix
<code>assetlinks.json</code> returns 301 redirect	Netlify redirect rule intercepting the path	Add explicit <code>200</code> rewrite rule in <code>_redirects</code>
<code>assetlinks.json</code> returns wrong Content-Type	Missing <code>_headers</code> rule	Add <code>Content-Type: application/json</code> in <code>_headers</code>
Instagram/Facebook link opens browser, not app	IAB blocks Universal Links — expected	Confirm JS fallback is firing <code>ecoatm://</code>
Deep link works in testing but not after release	Release keystore SHA-256 differs from debug	Update <code>assetlinks.json</code> with release cert fingerprint
App opens to home screen instead of correct screen	<code>onNewIntent</code> not implemented	Add <code>onNewIntent</code> override in <code>MainActivity</code>
App Links stop working after updating app	New signing cert introduced	Update SHA-256 in <code>assetlinks.json</code> and redeploy

13. Open Questions & Dependencies

#	Question	Owner	Blocking
1	Confirm production Android bundle ID (QA vs prod)	Mobile Engineering	<code>assetlinks.json</code> , manifest
2	Confirm branded short domain for SMS (<code>eco.atm</code> or similar)	Marketing / IT	SMS links
3	What is the Apple Team ID?	iOS Engineering	<code>apple-app-site-association</code>
4	What is the release keystore SHA-256 fingerprint?	Android Engineering	<code>assetlinks.json</code>
5	Is <code>links.ecoatm.com</code> the confirmed deep link domain?	DevOps / IT	All links, AASA, assetlinks

#	Question	Owner	Blocking
6	What are all the screen route names in the app?	Mobile Engineering	<code>DeepLinkHandler</code> routing
7	Does the ESP (Braze/Iterable/SFMC) need a native deep link integration?	Marketing Ops	Email links
8	Is there a push notification provider already configured (FCM/APNS)?	Mobile Engineering	Push payload format

This document was prepared as part of a deep linking spike for the ecoATM mobile application. All sample URLs use `links.ecoatm.com` as a placeholder — replace with the confirmed domain before implementation. Code samples are illustrative starting points and should be adapted to the app's existing navigation architecture.